ARM64 Cache Thrashing - Performance Bottleneck 1
Overview 1
ARM64 Cache Structure 1
Causes of Thrashing 1
1. PowerofTwo Strides 2
2. Excessive Lines per Set 2
3. Poor Data Alignment 2
Symptoms 2
Avoiding Thrashing 2
1. Change Stride 3
2. Add Padding 3
3. Adjust Data Layout 3
4. Use Prefetching 3
5. Use WriteThrough or NonCacheable Memory 3
Example: Thrashing vs. No Thrashing 3
Thrashing Pattern 4
Fixed Pattern 4
Generic Coding Approach to Avoid Cache Problems 4
1. Default Loop Pattern 4
Use unit stride whenever possible 5
Only introduce strides when algorithmically required 5
Avoid poweroftwo strides 5
2. Data Layout Rules 5
Prefer StructureofArrays (SoA) for hot numeric code 5
3. Padding Strategy 5
Generic padding rule 6
4. When to Think About Caching Explicitly 6
5. The Right Everyday Approach 6
Conclusion 7

ARM64 Cache Thrashing - Performance Bottleneck

Overview

Cache thrashing is a performance issue that occurs when repeated memory accesses map to the same cache set, causing continuous eviction and reload of cache lines. ARM64 processors, including Cortex-A73, use set-associative caches where each memory address maps to a specific cache set based on index bits. Thrashing happens when an access pattern repeatedly targets one or a small number of sets, exceeding the associativity limit.

ARM64 Cache Structure

- **Cache Line Size:** 64 bytes
- **Associativity:** 16-way (Cortex-A73 L1D)
- **Total Sets:** 32 sets
- **Index Bits:** Address bits [11:6]

Each memory access selects a cache set using the index bits. If multiple active lines map to the same set and exceed the associativity, thrashing occurs**. Goal is to** ensure that frequently accessed addresses do NOT map to the same cache set(index).

Causes of Thrashing

1. Power-of-Two Strides

Access patterns with strides that are multiples of the cache line size often keep index bits constant.

Example:

- Accessing `A[i]` with `i += 64` (stride = 256 bytes)
- Only bit 8 changes, leaving index bits mostly unchanged
- Accesses repeatedly hit the same set

2. Excessive Lines per Set

If more than 16 distinct lines map to the same set, the cache must evict older lines, causing thrashing.

3. Poor Data Alignment

Arrays aligned to cache set boundaries can amplify thrashing when combined with regular stride patterns.

Symptoms

- High L1 miss rate
- Increased L2 traffic
- Periodic performance drops
- Performance improves when stride or array size changes

Avoiding Thrashing

1. Change Stride

Use non-power-of-two strides to ensure index bits change.

Example:

```
for (i = 0; i < N; i += 63)
    sum += A[i];
```

- Stride = 252 bytes
- Index bits change
- Accesses spread across sets

2. Add Padding

Padding shifts the base address, altering index bits.

Recommended padding:

- **64 bytes** (one cache line) to guarantee index bit changes

3. Adjust Data Layout

Structure-of-arrays (SoA) layouts reduce set conflicts compared to array-of-structures (AoS).

4. Use Prefetching

Software prefetching improves locality and reduces eviction pressure.

5. Use Write-Through or Non-Cacheable Memory

For problematic regions, adjusting memory attributes can reduce dirty-line eviction storms.

Example: Thrashing vs. No Thrashing

Thrashing Pattern

```
for (i = 0; i < N; i += 64)
    sum += A[i];
```

- Stride = 256 bytes
- Index bits remain constant
- Accesses hit only 1–2 sets
- Thrashing occurs

Fixed Pattern

```
for (i = 0; i < N; i += 63)
    sum += A[i];
```

- Stride = 252 bytes
- Index bits change
- Accesses spread across sets
- No thrashing

Generic Coding Approach to Avoid Cache Problems

You should not think about cache sets, index bits, or associativity every time you write code. Instead, follow these **generic, reusable coding patterns** that naturally avoid thrashing and other cache issues.

1. Default Loop Pattern

Use unit stride whenever possible

```
for (i = 0; i < N; i++)
    process(A[i]);
```

Sequential access naturally uses spatial locality.

Only introduce strides when algorithmically required

If you must skip elements:

```
for (i = 0; i < N; i += step)
    process(A[i]);
```

Avoid power-of-two strides

Power-of-two strides often align with cache geometry.

2. Data Layout Rules

Prefer Structure-of-Arrays (SoA) for hot numeric code

```
// Instead of:
struct Point { float x, y, z; };
Point pts[N];

// Prefer:
float x[N], y[N], z[N];
```

SoA improves locality and reduces set conflicts.

3. Padding Strategy

Use padding only when profiling shows performance cliffs.

Generic padding rule

Add **64 bytes** (one cache line) to hot arrays:

```
#define PAD 16   // 16 floats = 64 bytes
float A[N + PAD];
```

This shifts index bits and breaks alignment.

4. When to Think About Caching Explicitly

You only need to think about cache geometry in these cases:

- Sharp performance drop at specific sizes or strides
- Inner loops of DSP/ML/graphics kernels
- Highly regular stride-based loops
- Multi-threaded loops accessing shared arrays

Otherwise, rely on the generic rules.

5. The Right Everyday Approach

Follow this checklist:

1. Write simple, sequential loops first.
2. Use SoA for hot numeric data.
3. Avoid power-of-two strides.
4. Only think about cache sets when profiling shows a cliff.
5. Fix cliffs with:
    a. Changing stride
    b. Adding 64-byte padding

    c. Chunking/tiling the work
6. Document cache-aware tuning in the code.

This approach keeps your code clean, portable, and fast without requiring constant cache analysis.

Conclusion

Understanding cache geometry and access patterns is essential for avoiding thrashing on ARM64 systems. By adjusting stride, adding padding, and following generic coding patterns, developers can significantly improve performance and reduce cache conflicts.